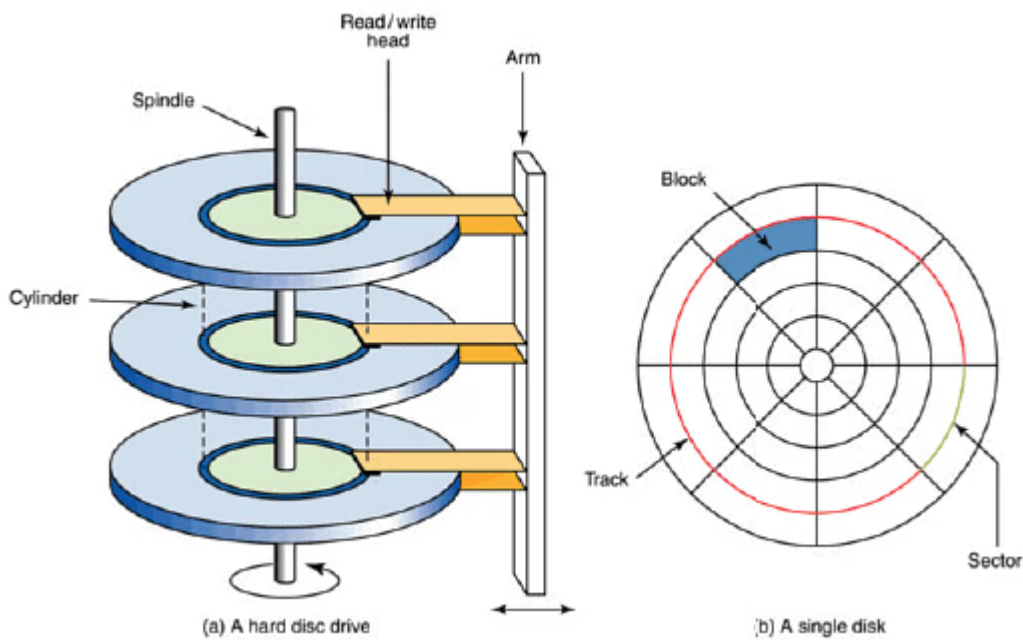


Links:

- [NoSQL](#)
- [documentation](#)
- [Postgres](#)
- [Screencasts Intro to FTS](#)

Memory Hierarchy



iMagnetic disk animation

SRAM → **Static** RAM DRAM → **Dynamic** RAM

1. **Latency** time to read 1st value from storage location (↓ is better)

Register < SRAM < DRAM < SSD < HDD

2. **Density** $\frac{\text{no. of bits stored}}{\text{cost} \times \text{unit area}}$ ↑

Regs < SRAM < DRAM < SSD < HDD

3. **Throughput** $\frac{\text{no. of bytes can be read}}{\text{second}}$ (↑ is better)

DRAM > SSD > HDD

- On-chip **registers**: **10-100B**
 - **SRAM (L1, L2, L3 cache)** **0.1-1 MB**
 - **DRAM** **0.1-10 GB** working memory

- solid-state disk **SSD** Flash **1-100 GB** ↑ high throughput
- Magnetic disk (**HDD**) **0.1-10 TB** ↑ High capacity

Cold storage

- **Amazon Glacier, Google, Azure Cold/Archive storage classes**
 - long term storage/backup Huge amount ❌ read very often
 - ↑ durability, ↓ cost
 - can tolerate ↑ read latency (<48Hr)
- **optical, magnetic, holographic**

DRAM > SSD > HDD (regs, SRAM limited capacity)

Bandwidth=(Clock Speed)×(Data Transfers per Clock)×(Bus Width in Bytes)×

iCores threading

Data search

$O(1)$ CONSTANT TIME ind of input size, excellent but ❌ even does through c
 $O(\log_2 N)$ grows slowly LOGARITHMIC with input, best in practical **Binary sea**
 $O(N)$ LINEAR - baseline **search in unsorted array/linked list**
 $O(N^k)$ POLYNOMIAL - ❌ good as input ↑
 $O(k^N)$ EXPONENTIAL very bad even for reasonably small inputs

Question 8 : 6406531173006

[View Solutions \(0\)](#)

Total Mark : 3.00 | Type : MCQ

Question 2 : 6406531021272

[View Solutions \(0\)](#)

Total Mark : 4.50 | Type : MCQ

You have a DRAM module with bus width of 64 bits, clock speed of 800 MHz, and operating in DDR (double-data-rate or two values per clock cycle) mode. What is the maximum memory bandwidth in Gigabytes per second achievable with this module if it is working in dual channel mode (two memory interfaces are used)?

OPTIONS :

☐ 51.2

☐ 102.4

☒ 25.6

☐ 12.8

Bandwidth = (Clock Speed) × (Data Transfers per Clock) × (Bus Width in Bytes) × (Channels)

$$\begin{aligned} &= 800 \times 10^6 \times 2 \times \frac{64}{8} \times 2 \\ &= \frac{25600 \times 10^6 \rightarrow \text{Mega}}{10^9 \rightarrow \text{Giga}} = 25.6 \end{aligned}$$

Mega $\rightarrow 10^6$
Giga $\rightarrow 10^9$
GBps

$$= \frac{25600}{1000} = 25.6$$

Question 15 : 640653856976

View Solutions (1)

Total Mark : 4.50 | Type : MCQ

You have a DRAM module with bus width of 64 bits, clock speed of 400 MHz, and operating in DDR (double-data-rate of two values per clock cycle) mode. What is the maximum memory bandwidth in Gigabytes per second achievable with this module if it is working in dual channel mode (two memory interfaces are used)?

OPTIONS :

☐ 51.2

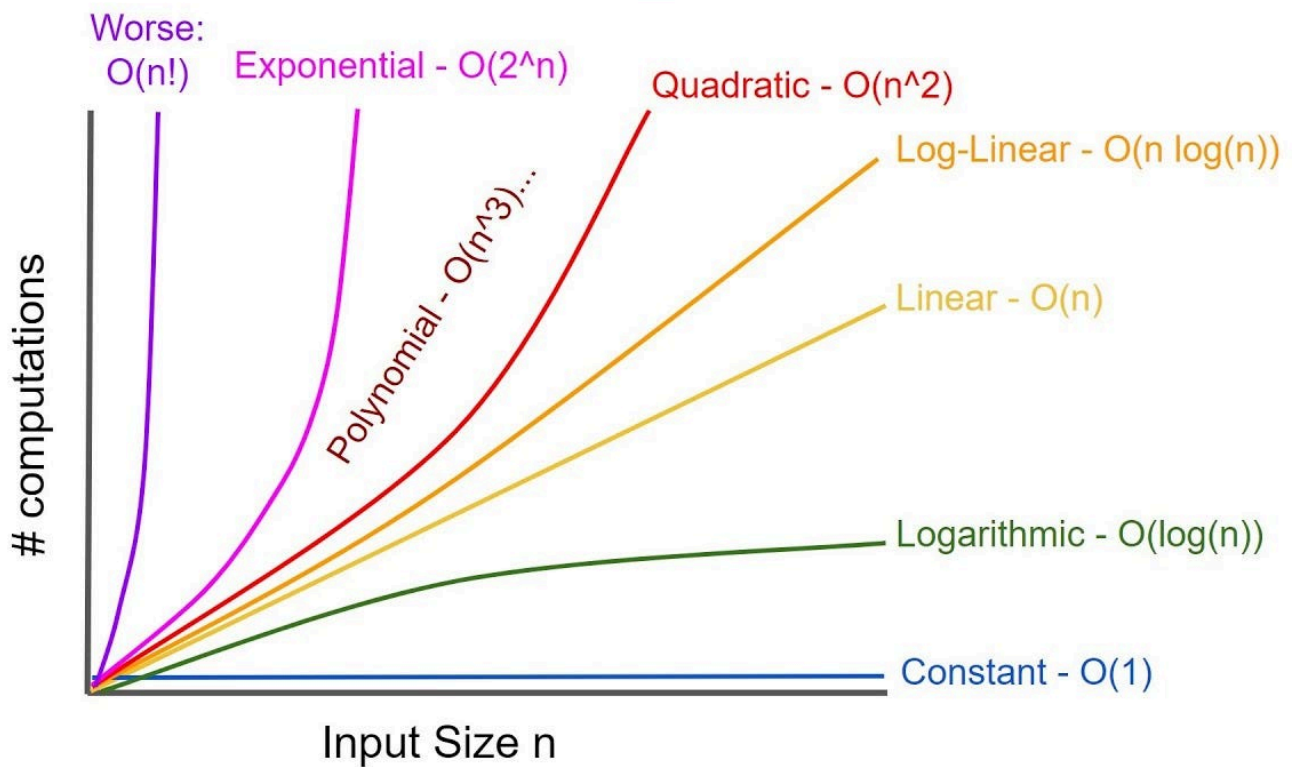
☐ 102.4

☐ 25.6

☒ 12.8

data rate

$$\text{Bandwidth} = \text{clock speed} \times \text{Data Transfer per clock} \times \text{Bus width (bytes)} \times \text{Channel}$$
$$= 400 \times 10^6 \times 2 \times \frac{64}{8} \times 2$$
$$= \frac{128 \times 10^8}{10^9} = 12.8$$

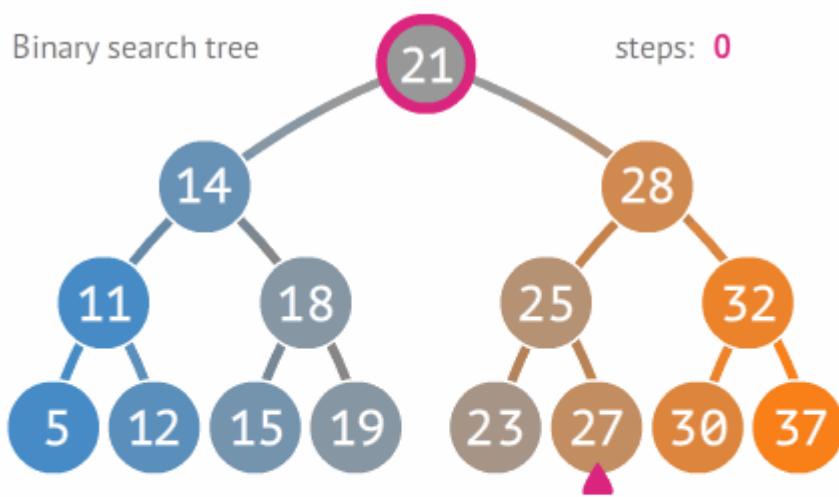


Binary Search tree

We need **BST** as sort by growth of tree but Self-balancing (tilt to 1 side grow downwards so **Red-Black, AVL, B-tree**):

- fixed size array
- maintaining sorted order $O(N)$
- append needs new size
- delete needs move all entries by 1 step
- Hash table if unique index $\rightarrow O(1)$

Binary search tree



Sorted array



www.penjee.com

Databases

- Tables with many columns → want to search quickly → ☒ proper **INDEX** ↔ sorted ver of column (same datatype short string/int/date)
- Hash-index ☒ `pattern%` ☒ `%pattern%` `like other_col`
 - in-memory tables
 - ☒ equality comparisons ☒ range
 - ☒ `ORDER BY`
 - ☒ partial key prefix

NoSQL

SQL vs. NoSQL: Comparison

ml4devs.com/datastores



SQL		NoSQL
 Relational	Model	Non-relational
Structured tables	Data	Semi-structured
Strict schema	Flexibility	Dynamic schema
ACID	Transactions	Mostly BASE, few ACID
Strong	Consistency	Eventual to Strong
Consistency prioritized	Availability	Basic Availability
Vertically by upgrading hardware	Scale	Horizontally by data partitioning

Key-Value



Dictionary or Hash Table

Wide Column



2-D Versioned Key-Value

Document



Nested Objects (XML, JSON, YAML)

Graph



Entity-Relationships

© Satish Chandra Gupta

CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/



scgupta.me

twitter.com/scgupta

linkedin.com/in/scgupta

ACID: Atomic, Consistent (might sacrifice → eventual), Isolated, Durable

BASE: Basically Available, Soft State, Eventually Consistent (high available replication) ❌ Financial ✅ Social media ✅ E-com non-fin parts

Document Databases

MongoDB, Couchbase, Amazon DocumentDB ❌ fixed size table (free-from nesting & structure)

- JSON k-v pairs: obj, records, structs, lists, arrays, maps, dates
- structured but each doc has own structure

```

{
    "books": [
        {
            "id": "01",
            "language": "Java",
            "author": "H. Javeson",
            "year": 2015
        },
        {
            "id": "07",
            "language": "C++",
            "edition": "second",
            "author": "E. Sepp",
            "price": 22.25
        },
    ]
}

```

•

key-value

python **dictionary**, C++ ordered map **DynamoDB, Voldermort, Scalaris, Redis, BerkeleyDB, Memcache**

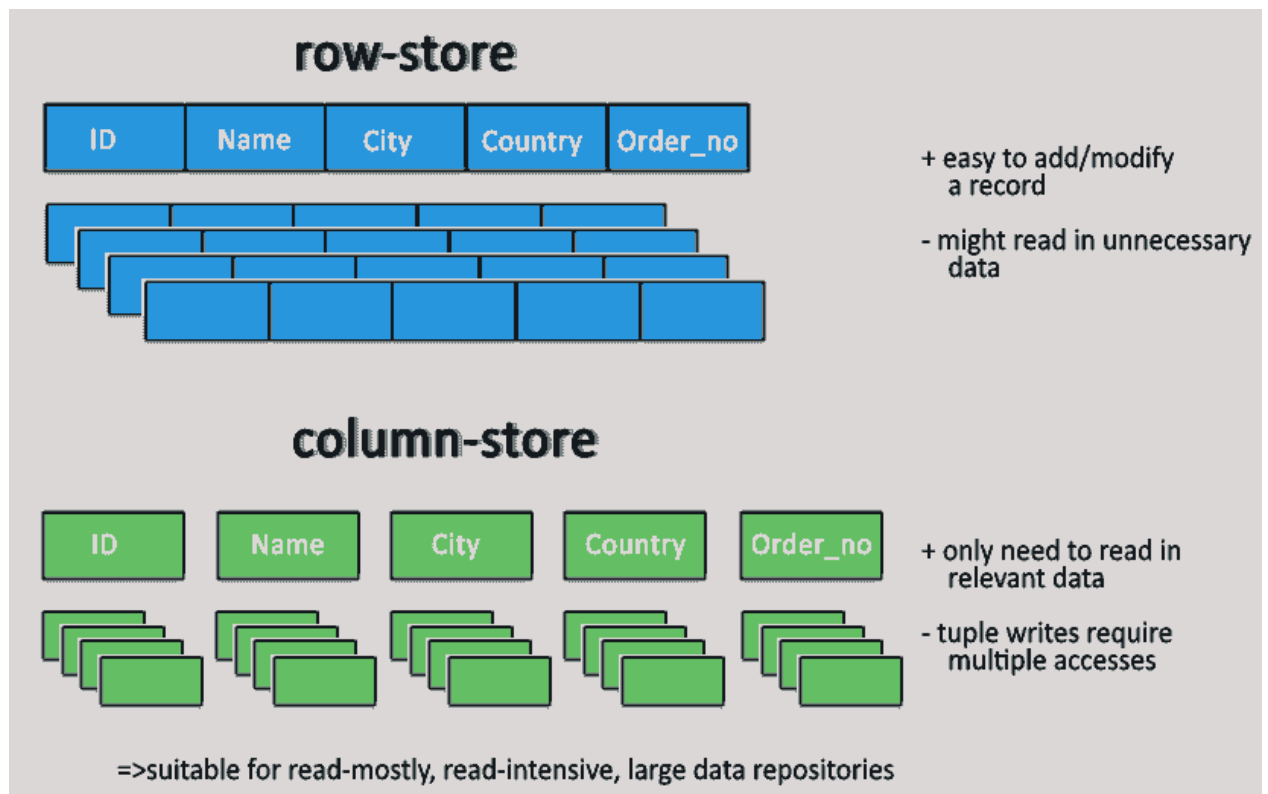
- Store using search trees or hash table, ≥ 1 attributes(1/multi-valued)
- ☒ key lookup ☒ range-type relation queries
- ☒ **in-memory** queries VERY FAST ☒ complex data objects

•

column stores

- ☒ Collections of column families(variable #cols)=
 (key, value) ; each row/record has ≥ 1 kv pairs **BigTable,**
 set of related cols
Cassandra, HBase

all sparse sorted values of given column family (in 1 disk) very FAST



Graphs

- $G = \overbrace{(V, E)}^{\text{k-v pair on both}}$ social-network outgoing degrees (relations),

$\downarrow$
 property/IDlabel/roles

$\downarrow$
 property/IDlabel/roles

weight of edges nodes **Path finding**

- OrientDB, Neo4J, Amazon Neptune, Info-grid

Time Series Databases

- optimized for time-stamped or time series data **log performance analysis, monitoring**
- **RRDTool, InfluxDB, Prometheus, elasticsearch, grafana**
- are tracked, monitored, down-sampled, & aggregated over time

- **Redundancy**

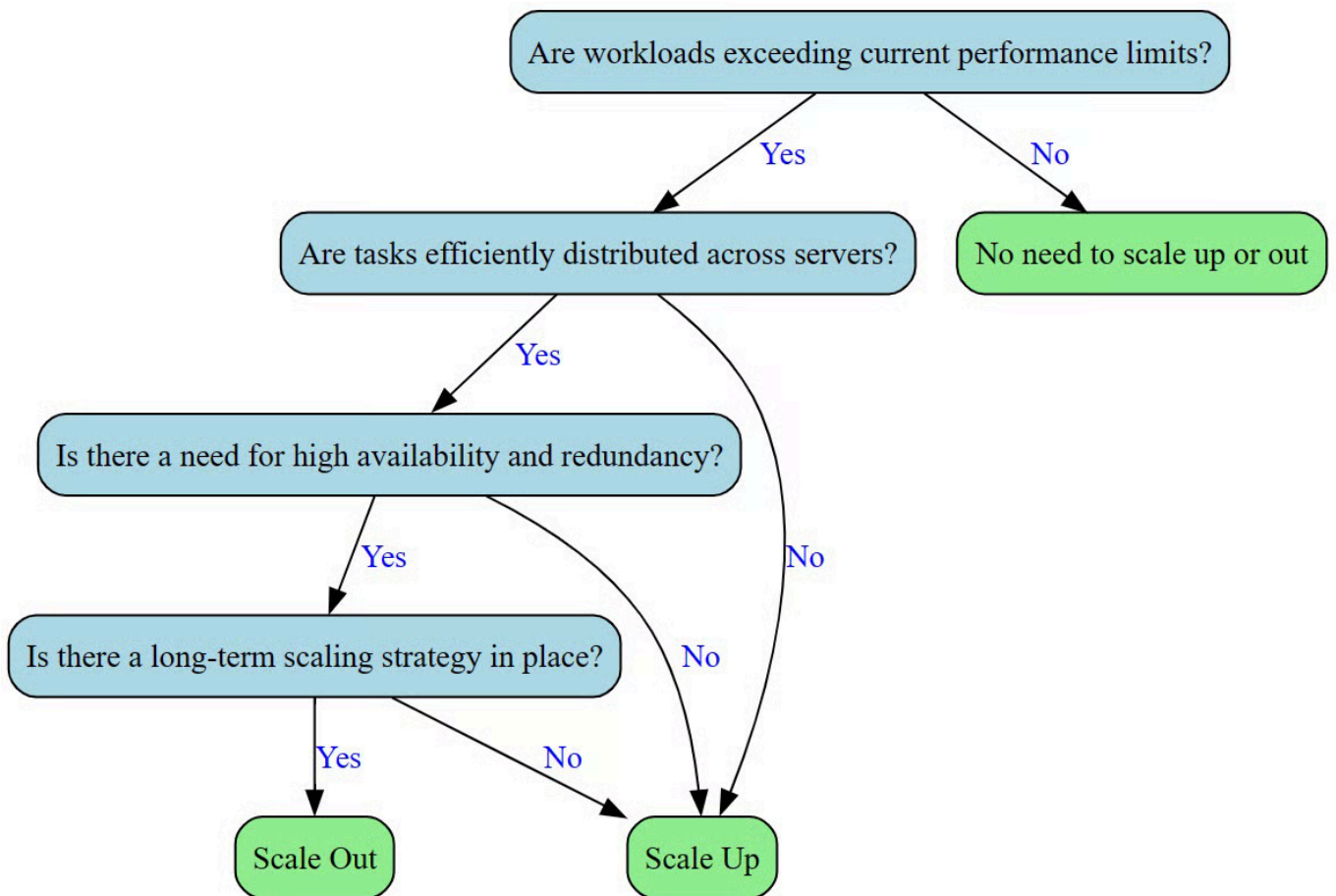
- multiple copies as backup if 1 master fails

- **Replication**

- **✗** backup but multiple sources of ~ data **server cluster in high speed network in ~ datacenter** for load balancing(**✗** overload) but geographically dist (Latency → ↓ consistency)

Scale-up vs Scale-out

- Scale-up
 - larger machine
 - ↑ RAM
 - faster network, processor
 - machine restart with each scale change
- Scale-out
 - multiple servers
 - hard to get consistency
 - cloud model auto scale-out
 - auto scale-up



Security

SQL injection: validation before send to DB query (even b4 HTML or Javascript validation)

- known frameworks ☒ servers with good track record, code audits

- Buffer overflows, input overflows (length of queries → client/server might crash) **google app engine**
- Update all patches on OS-network stack-server, encodings char sets

HTTPS

client ← secure sockets → server

- **Server certificate:** DNS verified by trusted 3rd party, difficult to spoof, maths properties very ↓ probability of mistakes match.
-  secure link for data transfer  validation/safety checks
- ↑ performance overhead or ↓ caching (intermediate proxies can't read packets)

Question 8 : 6406531173006

View Solutions (0)

Total Mark : 3.00 | Type : MCQ

A web server is handling requests and has an 8-core processor, 32 GB of RAM, and a network bandwidth of 500 Mbps. The application running on the server is capable of generating 100 HTML pages every second. Each of these pages includes an image file that is 500 KB in size. What is the maximum number of requests per second that the server can handle?

OPTIONS :

☒ 100 requests/sec☐ 125 requests/sec☐ 250 requests/sec☐ 400 requests/sec Check Answer

Your score : 0

$$\begin{aligned} & 500 \times 10^3 \times 8 \\ & \text{Bytes} \\ \text{Max requests} &= \frac{\text{Bandwidth}}{\text{Total no. of bits}} \\ &= \frac{500 \times 10^6}{500 \times 10^3 \times 8} \\ &= \frac{10^3}{8} = \frac{1000}{8} = 125 \text{ requests/sec} \end{aligned}$$

Question 2 : 6406531325139

 View Solutions (0)

Total Mark : 2.00 | Type : MCQ

You manage a datacenter where each server has a 10 Gigabit per second network connection and the facility's backbone link is also 10 Gbps. Your application is handling 10,000 requests per second, each with a 50 KB response, which translates to about 4 Gbps of required throughput. So, if each request involves minimal CPU processing, and the existing server has sufficient resources to handle the load. Which of the following scaling strategies is most appropriate to meet this demand?

OPTIONS :

- ☐ Scale up on a single server
- ☐ Scale out to multiple servers in the datacenter
- ☐ Both will produce similar results
- ☐ None of these



Check Answer

Your score : 0